Progressive Feature Engineering and Imbalanced Machine Learning for Credit Card Fraud Detection

First Author Name
Department or Faculty
Institution Name
City, Country
email@example.com

Second Author Name
Department or Faculty
Institution Name
City, Country
email@example.com

Abstract

Credit card fraud detection is an imbalanced tabular classification problem in which minority-class performance is more informative than overall accuracy. This study evaluates researcher-generated synthetic transaction data containing 499,985 records, including 62,500 fraudulent and 437,485 legitimate transactions, for a fraud ratio of 12.5004%. Five progressively enriched dataset versions are constructed from the same cleaned transaction population. Logistic Regression, Decision Tree, Random Forest, XGBoost, and CatBoost models are evaluated under No-SMOTE and SMOTE settings using a stratified 60/20/20 train-validation-test split, 5-fold cross-validation on the training split, and PR-AUC, implemented as average precision, for hyperparameter tuning. At the default threshold, the strongest result is obtained by v3 SMOTE Random Forest, with precision 0.998525, recall 0.758080, fraud-class F1 0.861846, and PR-AUC 0.855235. After validation-based threshold tuning, the strongest operating point is v5 No-SMOTE Random Forest at threshold 0.80, with precision 0.999156, recall 0.758000, and fraud-class F1 0.862030. The results indicate that progressive feature engineering improves fraud-class performance and that the benefit of SMOTE is threshold-dependent. The findings are limited by the synthetic dataset, absence of external validation, and leakage-risk proxy indicators.

Keywords - credit card fraud detection; imbalanced classification; feature engineering; SMOTE; Random Forest; XGBoost; CatBoost; PR-AUC

I. INTRODUCTION

A. Background and Motivation

Credit card fraud detection is commonly formulated as binary transaction classification, where fraudulent transactions are much less frequent than legitimate transactions but carry high analytic importance. In such settings, accuracy can be misleading because the majority class dominates aggregate error. Fraud-class precision, recall, F1, and PR-AUC are therefore central to evaluating model behavior [9]-[17].

This paper studies a controlled tabular machine-learning pipeline for fraud detection using a fixed synthetic transaction population. Fig. 1 summarizes the full workflow used to move from dataset preparation through model selection, test evaluation, and operating-threshold analysis.

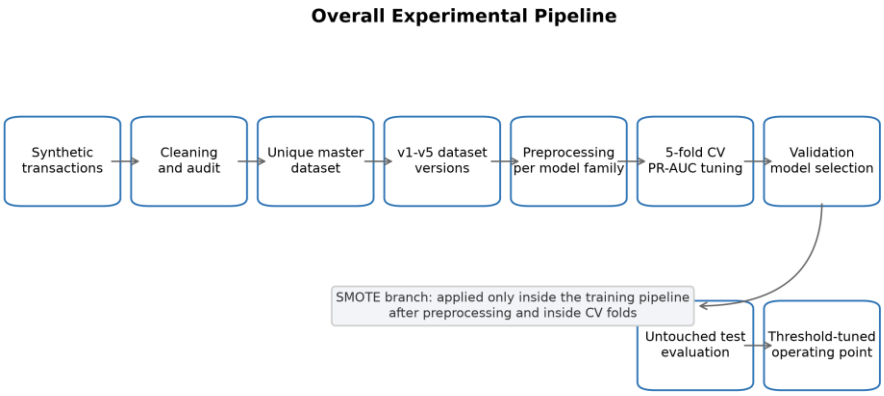

Fig. 1. Overall experimental pipeline.

B. Research Gap

Many fraud-detection studies compare classifiers or resampling strategies, but fewer isolate the effect of progressive feature enrichment while holding the transaction population fixed. Without that control, performance changes can be confounded by different row counts, class ratios, or duplicate transactions.

C. Contributions

1. A five-version progressive feature engineering design using a fixed synthetic transaction population.
2. A controlled comparison of five machine-learning models under No-SMOTE and SMOTE settings.
3. A leakage-aware splitting and SMOTE placement strategy that keeps resampling inside the training pipeline.
4. A default-threshold versus threshold-tuned comparison using validation-based operating-point selection.
5. A reproducibility-focused package of datasets, metrics, figures, tables, and audit reports.

The paper does not claim evaluation on institutional transaction records, live decisioning, or external generalization. Claims are restricted to the repository evidence generated from the cleaned synthetic dataset.

D. Paper Organization

Section II summarizes related work. Section III describes dataset construction and validation. Section IV details the methodology. Section V reports results. Section VI discusses implications. Section VII lists threats to validity. Section VIII provides data, ethics, funding, competing-interest, and author-contribution statements. Section IX concludes the paper.

II. RELATED WORK

A. Rule-Based Fraud Detection

Rule-based fraud detection encodes expert knowledge through manually specified thresholds and transaction patterns. These systems can be interpretable, but they require updating as fraud patterns and transaction behavior change [1], [2].

B. Machine Learning for Credit Card Fraud Detection

Prior credit-card fraud studies have evaluated linear models, tree methods, ensemble models, boosting, and sequence-oriented approaches [3]-[8]. This study focuses on tabular classifiers implemented in the repository: Logistic Regression, Decision Tree, Random Forest, XGBoost, and CatBoost.

C. Imbalanced Learning and SMOTE

Imbalanced learning methods address the tendency of classifiers to favor the majority class [9]-[14]. SMOTE creates synthetic minority-class samples during training [9]. In this study, SMOTE is evaluated only as a training-pipeline condition, not as a preprocessing step before splitting.

D. Feature Engineering for Tabular Fraud Detection

Feature engineering can convert raw transaction fields into more predictive indicators, including amount-derived, temporal, merchant-level, and card-level features [5], [29], [30]. The present study uses progressive dataset versions to evaluate how feature enrichment changes fraud-class performance.

E. Evaluation Metrics for Imbalanced Classification

PR-AUC and fraud-class F1 are emphasized because ROC-AUC and accuracy may obscure minority-class behavior in imbalanced settings [15]-[17]. The repository uses average precision for hyperparameter tuning and validation fraud-class F1 for model-family selection.

III. DATASET CONSTRUCTION AND VALIDATION

A. Dataset Generation and Cleaning

The dataset is researcher-generated synthetic transaction data. It is not claimed to be sourced from institutional card-issuer records. The raw merged provenance file contained 849,999 rows and 350,014 exact duplicate rows. That raw merged file is retained only for provenance and appendix-style audit use, not as an additional main experiment.

After cleaning and transaction-level de-duplication, the final master dataset contains 499,985 unique transactions. Fig. 2 summarizes the progressive version design.

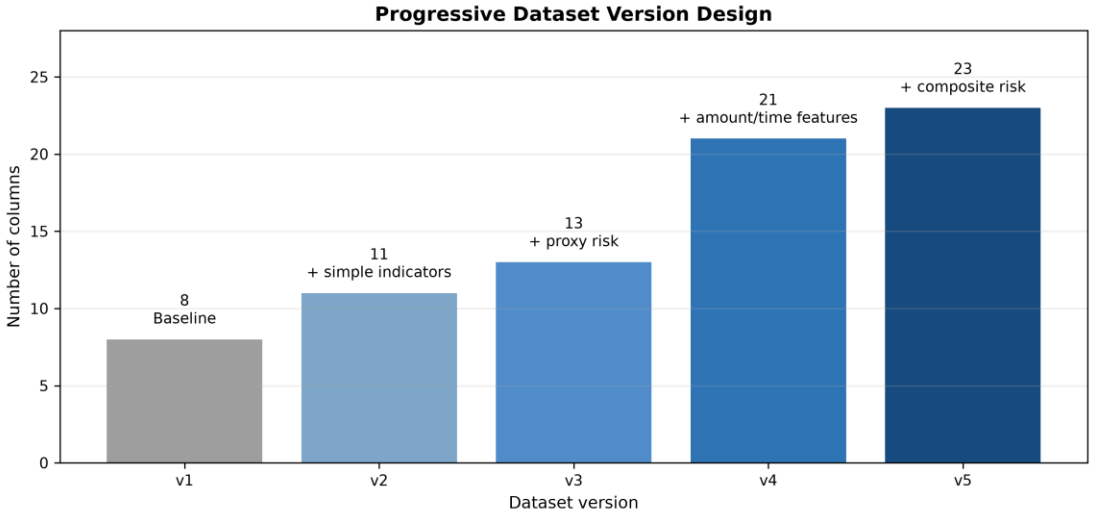

Fig. 2. Progressive dataset version design.

TABLE I
DATASET SUMMARY

Property	Value
Dataset file	data/research_master_dataset.csv
Dataset type	Researcher-generated synthetic transaction dataset
Task	Binary tabular classification
Target label	FraudFlag
Total transactions	499,985
Fraud transactions	62,500
Legitimate transactions	437,485
Fraud ratio	12.5004%
Missing values in final datasets	0
Exact duplicate rows in final datasets	0
Duplicate TransactionID values in final datasets	0
Language-processing features	Not present

B. Dataset Audit and Integrity Checks

The final datasets contain no missing values, no exact duplicate rows, and no duplicate TransactionID values according to the repository validation summary. TransactionID is a deterministic synthetic identifier used for reproducibility and overlap checks; it is excluded from modeling. FraudFlag is the binary target label and is also excluded from the feature matrix.

C. Class Distribution

The cleaned dataset contains 62,500 fraudulent transactions and 437,485 legitimate transactions. This corresponds to a fraud ratio of 12.5004%, as shown in Fig. 3.

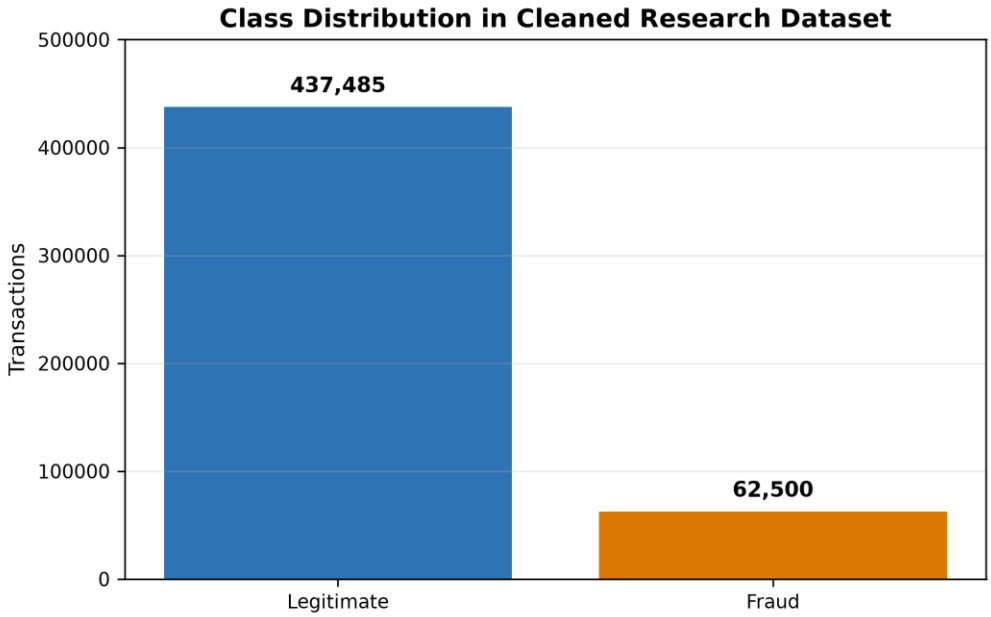

Fig. 3. Class distribution of the cleaned synthetic transaction dataset.

D. Progressive Dataset Versions

The five dataset versions use the same transaction population. This improves experimental control because differences across v1-v5 reflect feature availability rather than changes in row count, duplicate exposure, or class balance. Table II summarizes the version design.

TABLE II
PROGRESSIVE DATASET VERSIONS

Version	Rows	Columns	Added Feature Group	Purpose
v1	499,985	8	Baseline transaction, categorical, and time fields	Baseline comparator
v2	499,985	11	High-amount flag, night-transaction flag, subcategory	Assess simple risk indicators
v3	499,985	13	Merchant-risk and card-risk indicators	Assess proxy risk enrichment
v4	499,985	21	Amount and time engineered features	Assess advanced engineered predictors
v5	499,985	23	Night-high-amount flag and composite risk score	Assess full engineered feature set

E. Feature Exclusion and Leakage Controls

The active pipeline excludes FraudFlag, TransactionID, Time, DayOfWeek, Month, IsWeekend, and IsWeekendDerived from model features. MerchantRisk and CardRisk are retained only in v3+ as documented synthetic proxy indicators, and the manuscript treats them as leakage-risk features requiring caution. The synthetic nature of the data also limits external validity because observed performance may not transfer to independently collected transaction streams.

IV. METHODOLOGY

A. Experimental Pipeline

The experimental pipeline follows Fig. 1. Datasets are validated first, then each version is processed through preprocessing, model tuning, validation selection, untouched test reporting, SMOTE comparison, and threshold tuning.

TABLE III
EXPERIMENTAL CONFIGURATION

Component	Configuration
Split strategy	Stratified train/validation/test split
Split ratio	60/20/20
Training records	299,991
Validation records	99,997
Test records	99,997
Cross-validation	5-fold CV on training split
Hyperparameter tuning metric	PR-AUC / average precision
Model-family selection	Validation split
Final reporting	Untouched test split
SMOTE placement	Inside imblearn pipeline, after preprocessing, inside CV folds
Excluded model features	FraudFlag, TransactionID, Time, DayOfWeek, Month, IsWeekend, IsWeekendDerived

B. Preprocessing Strategy

The split is stratified 60/20/20, producing 299,991 training records, 99,997 validation records, and 99,997 test records. Logistic Regression uses imputation, one-hot encoding for categorical variables, and numeric scaling. Decision Tree, Random Forest, and XGBoost use imputation and frequency encoding for categorical variables. CatBoost follows the repository implementation, with leakage-safe feature engineering applied before CatBoost fitting and prediction.

C. Model Families

The implemented model families are Logistic Regression, Decision Tree, Random Forest, XGBoost, and CatBoost. Logistic Regression provides a linear baseline, while the tree-based and boosting models capture nonlinear thresholds and interactions in heterogeneous tabular features [18]-[22].

D. Hyperparameter Tuning and Model Selection

Hyperparameter tuning uses 5-fold cross-validation only on the training split, with PR-AUC implemented as average precision. Model-family selection is performed on the validation split using fraud-class F1 with tie-breaking based on PR-AUC, recall, precision, ROC-AUC, and lower training time. Test metrics are reported only after validation-based selection [23]-[26].

E. SMOTE Configuration

SMOTE is applied only inside the imblearn training pipeline, after preprocessing and inside the cross-validation folds. It is not applied before the train-validation-test split. This placement reduces split contamination risk and keeps validation/test sets representative of the original class distribution.

F. Threshold Optimization

Threshold optimization is performed after model training by selecting a fraud-probability threshold on validation predictions and evaluating the selected threshold on test predictions. This operating-point analysis is separate from the default-threshold model comparison.

G. Evaluation Metrics

The reported metrics are fraud-class precision, fraud-class recall, fraud-class F1, ROC-AUC, and PR-AUC. Fraud-class F1 and PR-AUC are emphasized because the minority class is the primary analytic target.

V. EXPERIMENTAL RESULTS

A. Default-Threshold Results Without SMOTE

Table IV reports the selected default-threshold No-SMOTE model for each dataset version. Performance increases sharply from v1-v2 to v3-v5, indicating that the risk-enriched and engineered feature sets provide stronger fraud-class separability.

TABLE IV
BEST DEFAULT-THRESHOLD RESULTS WITHOUT SMOTE

Version	Best Model	Precision	Recall	Fraud-Class F1	PR-AUC
v1	XGBoost	0.8062	0.7502	0.7772	0.8268
v2	XGBoost	0.8094	0.7500	0.7786	0.8274
v3	CatBoost	0.9219	0.7739	0.8414	0.8558
v4	DecisionTree	0.9537	0.7678	0.8507	0.8337
v5	RandomForest	0.9418	0.7713	0.8480	0.8561

B. Default-Threshold Results With SMOTE

Table V reports the selected default-threshold SMOTE model for each version. The strongest default-threshold result is v3 SMOTE Random Forest, with precision 0.998525, recall 0.758080, fraud-class F1 0.861846, and PR-AUC 0.855235.

TABLE V
BEST DEFAULT-THRESHOLD RESULTS WITH SMOTE

Version	Best Model	Precision	Recall	Fraud-Class F1	PR-AUC
v1	DecisionTree	0.9760	0.7040	0.8180	0.8237
v2	RandomForest	0.9527	0.7153	0.8171	0.8265
v3	RandomForest	0.9985	0.7581	0.8618	0.8552
v4	CatBoost	0.9998	0.7570	0.8616	0.8564
v5	CatBoost	0.9999	0.7571	0.8617	0.8558

C. Impact of Progressive Feature Engineering

Fig. 4 shows that the largest gain occurs from v2 to v3. Versions v3-v5 are practically close, so differences below 0.001 should not be overstated without repeated-seed or uncertainty analysis.

Fig. 4. Fraud-class F1 comparison across dataset versions.

D. Impact of SMOTE

SMOTE improves default-threshold fraud-class F1 in all five versions. However, it does not improve recall in every selected model. Fig. 5, Fig. 6, and Fig. 7 compare precision, recall, and PR-AUC under No-SMOTE and SMOTE settings.

Fig. 5. Fraud precision comparison across dataset versions.

Fig. 6. Fraud recall comparison across dataset versions.

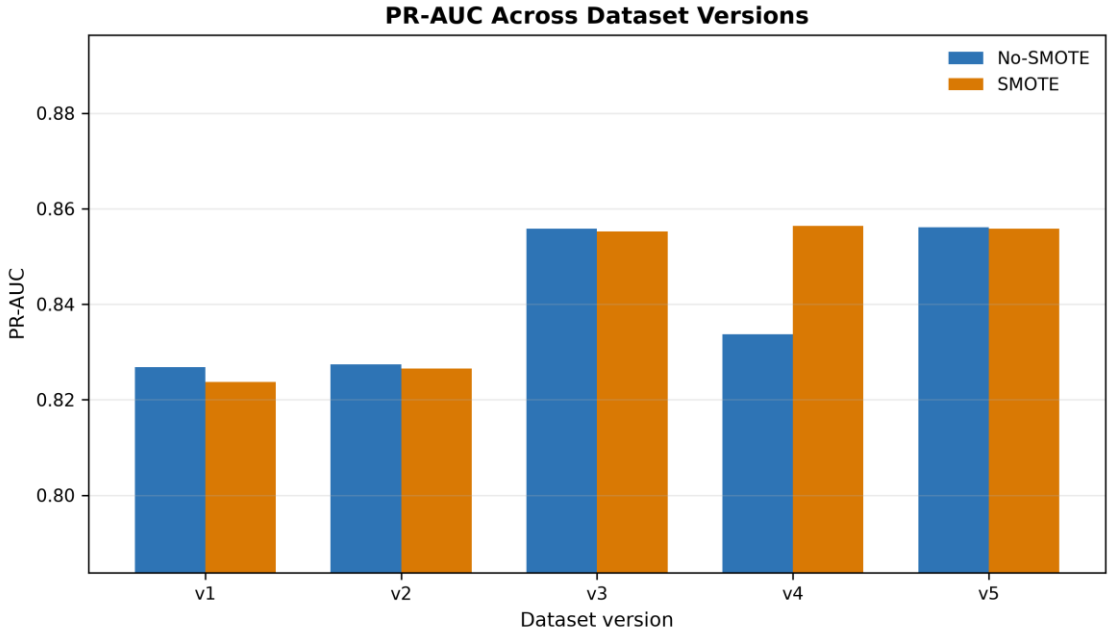

Fig. 7. PR-AUC comparison across dataset versions.

E. Threshold-Tuned Operating Points

Table VI reports validation-selected thresholds and test-set F1 values. Threshold tuning removes most of the practical SMOTE advantage: the best tuned F1 values differ by less than 0.001 across the strongest settings.

TABLE VI
THRESHOLD-TUNED COMPARISON

Version	No-SMOTE Model	No-SMOTE Threshold	No-SMOTE F1	SMOTE Model	SMOTE Threshold	SMOTE F1
v1	RandomForest	0.80	0.819141	CatBoost	0.44	0.818983
v2	XGBoost	0.83	0.818759	XGBoost	0.43	0.818828
v3	XGBoost	0.82	0.861660	XGBoost	0.51	0.861611
v4	DecisionTree	0.76	0.861826	XGBoost	0.47	0.861443
v5	RandomForest	0.80	0.862030	XGBoost	0.45	0.861520

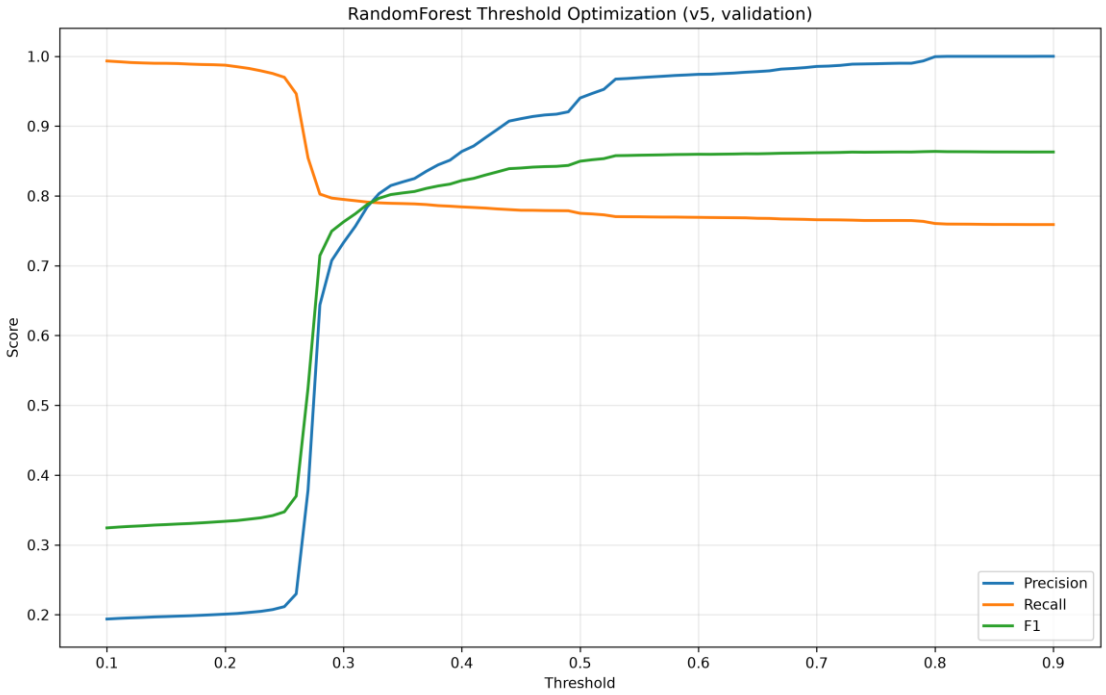

Fig. 8. Threshold optimization curve for the v5 No-SMOTE Random Forest model.

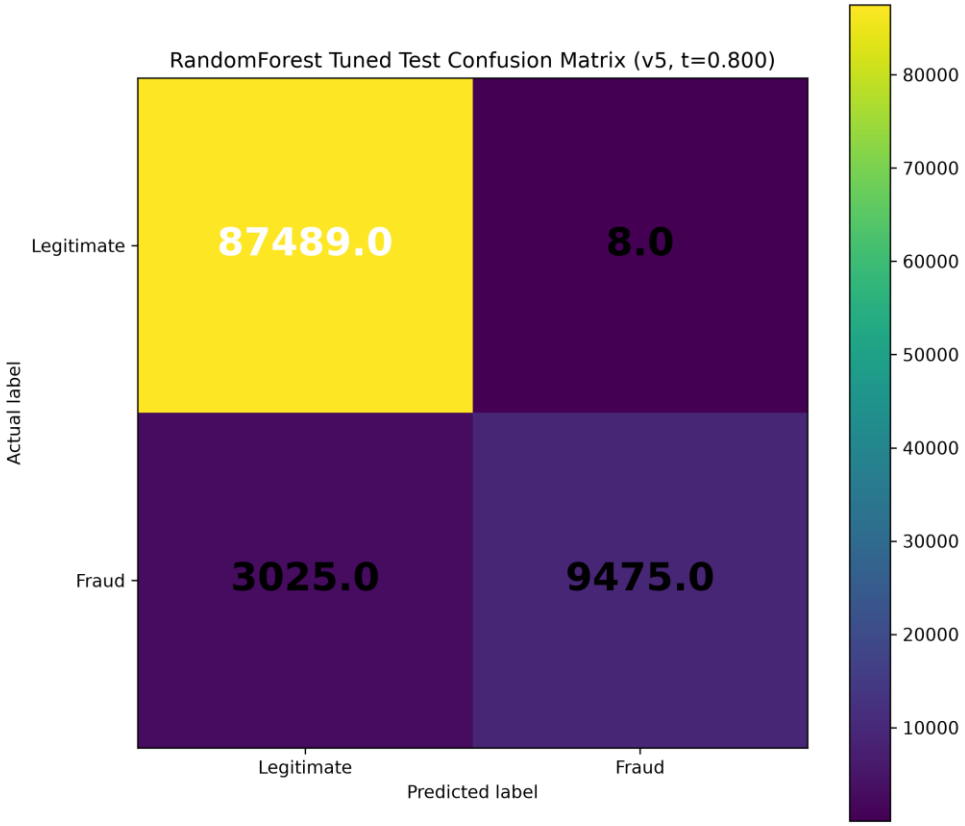

Fig. 9. Tuned test confusion matrix for the v5 No-SMOTE Random Forest model at threshold 0.80.

F. Best Model Interpretation

The strongest threshold-tuned operating point is v5 No-SMOTE Random Forest at threshold 0.80, with precision 0.999156, recall 0.758000, and fraud-class F1 0.862030. This result is only marginally higher than other strong tuned settings, so the interpretation should emphasize practical similarity among v3-v5 rather than a broad ranking claim.

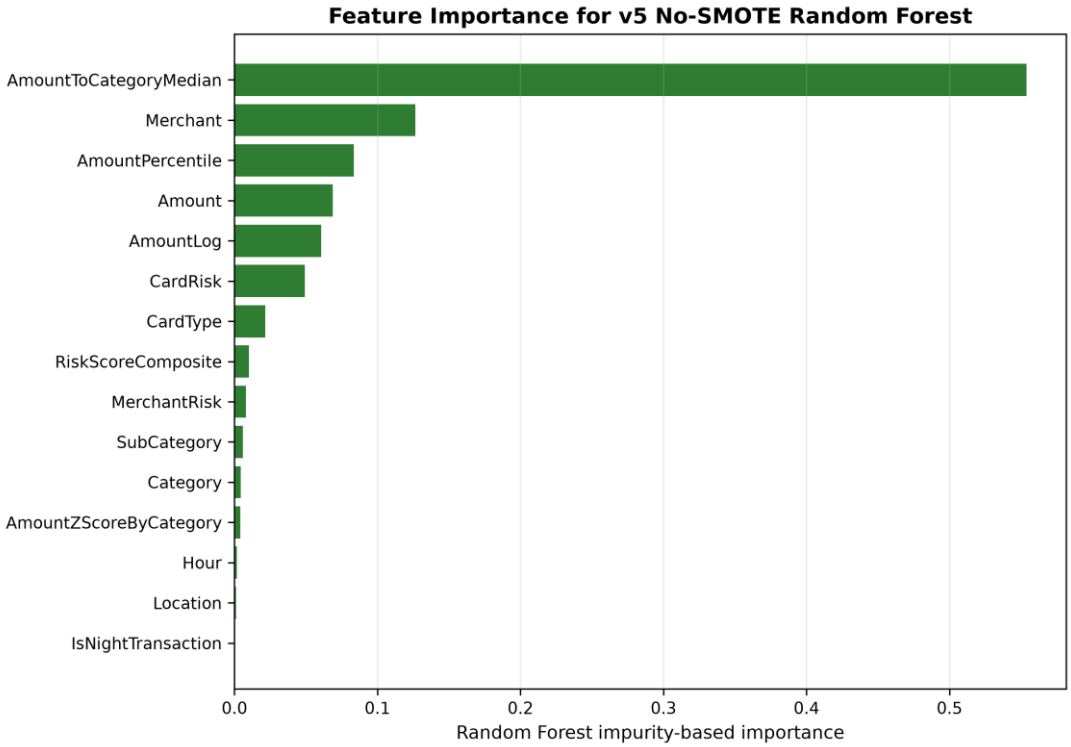

Fig. 10. Impurity-based feature importance for the selected v5 No-SMOTE Random Forest model.

TABLE VII
SUMMARY OF MAIN FINDINGS

Finding	Evidence	Interpretation
Progressive feature engineering improves fraud-class performance	v3-v5 outperform v1-v2 in fraud-class F1	Feature enrichment changes minority-class separability.
SMOTE improves default-threshold fraud-class F1	SMOTE selected models outperform no-SMOTE selected models in all five versions	The benefit is threshold-dependent.
Threshold tuning removes most practical SMOTE advantage	Best tuned F1 values differ by less than 0.001 across setups	Operating-point selection is central to deployment interpretation.
Tree-based methods dominate	Random Forest, XGBoost, CatBoost, and Decision Tree are selected in final best-model summaries	Nonlinear tabular interactions are important in this dataset.
Logistic Regression underperforms	No final best model is Logistic Regression	Linear baseline is less competitive for the engineered feature space.
Strongest versions are practically close	v3-v5 default and tuned results are close	Do not overstate the small differences among v3-v5.
External validity is limited	No external validation dataset is present	Results apply to the synthetic transaction dataset only.

VI. DISCUSSION

A. Interpretation of Findings

The results support a bounded claim: progressive tabular feature engineering improves fraud-class performance on the synthetic transaction dataset. The largest performance change occurs when proxy risk features are introduced in v3, suggesting that merchant- and card-level indicators carry strong predictive information in this generated population.

B. SMOTE Versus Threshold Optimization

SMOTE improves default-threshold F1, mainly by changing the precision-recall tradeoff of the selected models. Once thresholds are selected on validation predictions, No-SMOTE and SMOTE operating points become practically close. This shows that threshold policy can matter as much as resampling when reporting fraud-detection performance.

C. Practical Implications

The study indicates that applied fraud-analysis workflows should report minority-class metrics, explicitly document feature provenance, and evaluate threshold policies. Tree-based models likely perform well because they capture nonlinear interactions among amount, merchant, card, category, location, and time-derived variables.

D. Reproducibility Considerations

The repository provides fixed dataset versions, training outputs, validation/test predictions, threshold outputs, figures, and audit reports. Future external validation should test whether the same feature groups remain useful on independently collected transaction data.

VII. THREATS TO VALIDITY

A. Synthetic Data Limitation

Threat: the dataset is synthetic and researcher-generated. Impact: results may reflect the data-generation process rather than independently observed fraud behavior. Mitigation: the manuscript discloses the source type and restricts conclusions to the repository evidence; external validation remains future work.

B. External Validation Limitation

Threat: no external dataset is present. Impact: generalization beyond the cleaned master population is unknown. Mitigation: the study uses stratified train-validation-test splits and untouched test reporting, but independent validation remains required.

C. Leakage-Risk Features

Threat: MerchantRisk and CardRisk may encode target-proximal information. Impact: the v3-v5 gains could overestimate performance if equivalent indicators are unavailable at prediction time. Mitigation: the manuscript labels these as proxy risk indicators and interprets v3-v5 cautiously.

D. Threshold Dependence

Threat: performance depends on the selected probability threshold. Impact: default-threshold conclusions may differ from threshold-tuned conclusions. Mitigation: the study reports both settings and uses validation-based threshold selection before test reporting.

E. Absence of Statistical Uncertainty Estimates

Threat: the repository does not contain repeated-seed experiments, confidence intervals, bootstrap estimates, or formal significance tests. Impact: small differences among v3-v5 may be noise. Mitigation: the manuscript avoids overstating differences below 0.001 and recommends robustness analysis before submission.

F. Documentation Drift

Threat: legacy scripts and stale outputs remain in the repository. Impact: readers could confuse older artifacts with the final research-clean workflow. Mitigation: the manuscript and package cite only the active datasets, scripts, and result folders listed in the audit reports.

VIII. DATA AVAILABILITY, ETHICS, FUNDING, AND COMPETING INTERESTS

A. Data Availability

The datasets and code artifacts used in this study are available from the corresponding author or repository upon reasonable request, subject to repository release decisions.

B. Ethics Statement

This study uses researcher-generated synthetic transaction data and does not use personally identifiable customer information or live bank records.

C. Funding Statement

No external funding was reported for this study.

D. Competing Interests

The authors declare no competing interests.

E. Author Contributions

Author contributions will be completed before submission according to the target venue requirements.

IX. CONCLUSION AND FUTURE WORK

This study evaluated five progressively enriched synthetic credit-card transaction datasets using Logistic Regression, Decision Tree, Random Forest, XGBoost, and CatBoost under No-SMOTE and SMOTE settings. The strongest default-threshold result was v3 SMOTE Random Forest, with precision 0.998525, recall 0.758080, fraud-class F1 0.861846, and PR-AUC 0.855235. The strongest threshold-tuned operating point was v5 No-SMOTE Random Forest at threshold 0.80, with precision 0.999156, recall 0.758000, and fraud-class F1 0.862030.

The main finding is that progressive feature engineering improves fraud-class performance and that the benefit of SMOTE is threshold-dependent. The study remains limited by synthetic data, no external validation, leakage-risk proxy indicators, and missing statistical robustness analysis. Future work should add independent validation, repeated-seed experiments, uncertainty estimates, and cost-sensitive threshold selection.

REFERENCES

[1] R. J. Bolton and D. J. Hand, "Statistical fraud detection: A review," Statistical Science, vol. 17, no. 3, pp. 235-255, 2002, doi: 10.1214/ss/1042727940.

[2] E. W. T. Ngai, Y. Hu, Y. H. Wong, Y. Chen, and X. Sun, "The application of data mining techniques in financial fraud detection: A classification framework and an academic review of literature," Decision Support Systems, vol. 50, no. 3, pp. 559-569, 2011, doi: 10.1016/j.dss.2010.08.006.

[3] S. Bhattacharyya, S. Jha, K. Tharakunnel, and J. C. Westland, "Data mining for credit card fraud: A comparative study," Decision Support Systems, vol. 50, no. 3, pp. 602-613, 2011, doi: 10.1016/j.dss.2010.08.008.

[4] C. Whitrow, D. J. Hand, P. Juszczak, D. Weston, and N. M. Adams, "Transaction aggregation as a strategy for credit card fraud detection," Data Mining and Knowledge Discovery, vol. 18, no. 1, pp. 30-55, 2009, doi: 10.1007/s10618-008-0116-z.

[5] A. C. Bahnsen, D. Aouada, A. Stojanovic, and B. Ottersten, "Feature engineering strategies for credit card fraud detection," Expert Systems with Applications, vol. 51, pp. 134-142, 2016, doi: 10.1016/j.eswa.2015.12.030.

[6] A. Dal Pozzolo, O. Caelen, R. A. Johnson, and G. Bontempi, "Calibrating probability with undersampling for unbalanced classification," in Proc. IEEE Symposium Series on Computational Intelligence, 2015.

[7] J. Jurgovsky et al., "Sequence classification for credit-card fraud detection," Expert Systems with Applications, vol. 100, pp. 234-245, 2018, doi: 10.1016/j.eswa.2018.01.037.

[8] C. Phua, V. Lee, K. Smith, and R. Gayler, "A comprehensive survey of data mining-based fraud detection research," arXiv:1009.6119, 2010.

[9] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," Journal of Artificial Intelligence Research, vol. 16, pp. 321-357, 2002, doi: 10.1613/jair.953.

[10] H. He and E. A. Garcia, "Learning from imbalanced data," IEEE Transactions on Knowledge and Data Engineering, vol. 21, no. 9, pp. 1263-1284, 2009, doi: 10.1109/TKDE.2008.239.

[11] N. Japkowicz and S. Stephen, "The class imbalance problem: A systematic study," Intelligent Data Analysis, vol. 6, no. 5, pp. 429-449, 2002.

[12] M. Galar, A. Fernandez, E. Barrenechea, H. Bustince, and F. Herrera, "A review on ensembles for the class imbalance problem: Bagging-, boosting-, and hybrid-based approaches," IEEE Transactions on Systems, Man, and Cybernetics, Part C, vol. 42, no. 4, pp. 463-484, 2012, doi: 10.1109/TSMCC.2011.2161285.

[13] P. Branco, L. Torgo, and R. P. Ribeiro, "A survey of predictive modeling on imbalanced domains," ACM Computing Surveys, vol. 49, no. 2, pp. 1-50, 2016, doi: 10.1145/2907070.

[14] C. Elkan, "The foundations of cost-sensitive learning," in Proc. International Joint Conference on Artificial Intelligence, 2001, pp. 973-978.

[15] J. Davis and M. Goadrich, "The relationship between Precision-Recall and ROC curves," in Proc. International Conference on Machine Learning, 2006, pp. 233-240, doi: 10.1145/1143844.1143874.

[16] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," PLOS ONE, vol. 10, no. 3, 2015, doi: 10.1371/journal.pone.0118432.

[17] T. Fawcett, "An introduction to ROC analysis," Pattern Recognition Letters, vol. 27, no. 8, pp. 861-874, 2006, doi: 10.1016/j.patrec.2005.10.010.

[18] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5-32, 2001, doi: 10.1023/A:1010933404324.

[19] J. R. Quinlan, "Induction of decision trees," Machine Learning, vol. 1, no. 1, pp. 81-106, 1986, doi: 10.1007/BF00116251.

[20] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 785-794, doi: 10.1145/2939672.2939785.

[21] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in Advances in Neural Information Processing Systems, 2018.

[22] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[23] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," Journal of Machine Learning Research, vol. 13, pp. 281-305, 2012.

[24] R. Kohavi, "A study of cross-validation and bootstrap for accuracy estimation and model selection," in Proc. International Joint Conference on Artificial Intelligence, 1995, pp. 1137-1145.

[25] G. C. Cawley and N. L. C. Talbot, "On over-fitting in model selection and subsequent selection bias in performance evaluation," Journal of Machine Learning Research, vol. 11, pp. 2079-2107, 2010.

[26] S. Varma and R. Simon, "Bias in error estimation when using cross-validation for model selection," BMC Bioinformatics, vol. 7, article 91, 2006, doi: 10.1186/1471-2105-7-91.

[27] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," ACM Transactions on Knowledge Discovery from Data, vol. 6, no. 4, pp. 1-21, 2012, doi: 10.1145/2382577.2382579.

[28] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," Patterns, vol. 4, no. 9, 2023, doi: 10.1016/j.patter.2023.100804.

[29] I. Guyon and A. Elisseeff, "An introduction to variable and feature selection," Journal of Machine Learning Research, vol. 3, pp. 1157-1182, 2003.

[30] M. Kuhn and K. Johnson, Applied Predictive Modeling. New York, NY, USA: Springer, 2013.